

Arquitetura do frontend

Visão geral

O app é uma SPA React servida pelo Vite, sem framework de UI (Tailwind, MUI etc.) — toda a aparência vem de classes CSS compartilhadas definidas em `src/index.css` (ver [design-system.md](#) para a identidade visual). A comunicação com o banco é feita 100% via [Supabase JS SDK](#), nunca por uma API própria — as regras de autorização vivem no banco (RLS + RPCs `security definer`), não no frontend. O frontend só reflete essas regras na interface (esconder botões, redirecionar telas) para dar feedback ao usuário; a garantia real está no Postgres.

Ponto de entrada e providers

```
main.tsx
App.tsx
  ThemeProvider      (contexts/ThemeContext.tsx) - tema claro/escuro
  BrowserRouter
    AuthProvider      (contexts/AuthContext.tsx) - sessão Supabase + perfil
    AppRoutes         (routes/AppRoutes.tsx)
```

- **ThemeProvider** fica por fora de tudo porque a tela de login também precisa saber o tema (pra escolher a logo certa) antes de qualquer autenticação acontecer.
- **AuthProvider** assina `supabase.auth.onAuthStateChange` e mantém dois pedaços de estado: `session` (do Supabase Auth) e `profile` (a linha correspondente em `app_profiles`, com nome/papel/status). Ele expõe `signIn`, `signUp`, `signOut`, `requestPasswordReset` e `updatePassword` — ver [src/contexts/AuthContext.tsx](#).

Rotas e guards

Definidas em [src/routes/AppRoutes.tsx](#):

- **Públicas** (`/login`, `/cadastro`, `/esqueci-senha`): envolvidas por `PublicOnlyRoute`, que redireciona pra dentro do app se já houver sessão.
- **/redefinir-senha**: fora do `PublicOnlyRoute` de propósito — usa uma sessão temporária de recuperação de senha do Supabase, que tecnicamente já é uma "sessão".
- **Protegidas** (dashboard, notebooks, acessórios, telefonia, administração, auditoria): envolvidas por `ProtectedRoute` ([src/routes/ProtectedRoute.tsx](#)), que decide o que mostrar de acordo com o estado do perfil:
 1. sessão carregando → tela de "Carregando..."
 2. sem sessão → redireciona pra `/login`
 3. sessão sem perfil ainda (trigger do banco pode levar um instante) → "Preparando seu perfil..."
 4. perfil com `status !== 'APPROVED'` → `AccountStatusPage` (explica se está pendente, rejeitado ou bloqueado)
 5. aprovado → renderiza `AppLayout` (sidebar + header) com a página da rota dentro

Nenhuma rota filtra por **papel** (USER/ADMIN/MASTER) no React Router — quem faz isso é cada página individualmente (ex.: só mostra o botão "Novo notebook" se `canManage`) e, de forma definitiva, a RLS do banco. Um USER que force a URL de uma tela de admin não consegue gravar nada — as policies de `insert` / `update` dessas tabelas exigem `is_admin_or_master()`.

Fluxo de autenticação e aprovação

1. Usuário se cadastra (`SignupPage` → `supabase.auth.signUp`).

- O trigger `handle_new_user` (banco) cria automaticamente uma linha em `app_profiles` com `status = 'PENDING'` e `role = 'USER'` — **exceto** para o e-mail configurado como bootstrap MASTER, que já nasce `APPROVED` / `MASTER` (ver [banco-de-dados.md](#)).
- `ProtectedRoute` mostra `AccountStatusPage` enquanto o status não vira `APPROVED`.
- Um MASTER aprova, rejeita ou bloqueia pela tela **Solicitações de acesso** (`pages/admin/AccessRequestsPage.tsx`), que chama as RPCs `approve_user` / `reject_user` / `block_user` — nunca um `update` direto na tabela (não existe policy de update para `app_profiles`; é assim que o banco garante que ninguém promove o próprio perfil).
- Trocar o papel de alguém (`USER` ↔ `ADMIN` ↔ `MASTER`) usa a RPC `change_user_role`, que bloqueia remover o último MASTER do sistema e alterar a própria role.

Camada de serviços

Cada arquivo em `src/services/` concentra as chamadas Supabase de uma entidade (`notebooks.ts`, `accessories.ts`, `phoneLines.ts`, `employees.ts`, `adminUsers.ts`, `auditLogs.ts`, `telephony.ts`, `invoiceUploads.ts`, `referenceData.ts`, `dashboard.ts`). Regra do projeto: **páginas e componentes nunca importam supabase diretamente para ler/gravar dados de domínio** — sempre passam por uma função de serviço. Isso mantém queries e mapeamentos de erro num lugar só por entidade, em vez de espalhados pela UI.

`adminUsers.ts` é diferente dos demais: em vez de `supabase.from( ... )`, ele chama a Edge Function `admin-users` (`supabase.functions.invoke('admin-users', ...)`), porque criar usuário, resetar senha e excluir conta exigem a Service Role Key, que só existe no servidor (ver [credenciais-e-seguranca.md](#)).

Páginas

Rota	Arquivo	Quem acessa
/	<code>pages/DashboardPage.tsx</code>	Todos aprovados
/notebooks	<code>pages/NotebooksPage.tsx</code>	Todos aprovados (edição só ADMIN/MASTER)
/acessorios	<code>pages/AccessoriesPage.tsx</code>	idem
/telefonica	<code>pages/PhoneLinesPage.tsx</code>	idem — inclui <code>TelephonyDashboardPanel</code> e <code>InvoiceUploadsPanel</code>
/administracao/solicitacoes	<code>pages/admin/AccessRequestsPage.tsx</code>	ADMIN/MASTER (ações de aprovação só MASTER)
/administracao/colaboradores	<code>pages/admin/EmployeesPage.tsx</code>	ADMIN/MASTER
/administracao/categorias , /departamentos , /localidades , /operadoras	<code>pages/admin/ReferenceListAdminPage.tsx</code> (uma instância parametrizada por tabela) + <code>CategoriesPage</code> / <code>DepartmentsPage</code> / <code>LocationsPage</code> / <code>CarriersPage</code>	ADMIN/MASTER
/auditoria	<code>pages/admin/AuditLogPage.tsx</code>	Só MASTER

Dashboard: como os indicadores são calculados

`services/dashboard.ts` faz 5 leituras em paralelo (notebooks com ciclo de vida via a view `notebooks_with_lifecycle`, acessórios, linhas, categorias e localidades) e agrega tudo **no cliente** (contagem por status/categoria/localidade). É uma escolha deliberada: com a base atual (centenas de linhas, não milhares), isso é mais simples de manter do que criar uma RPC de agregação no banco, sem custo de performance perceptível. Se a base crescer muito, esse é o primeiro lugar a otimizar (mover a agregação pra uma RPC/view materializada).

O ciclo de vida de notebook (`TROCAR` / `ATENCAO` / `DENTRO_DA_VIDA_UTIL`) não é uma coluna — é calculado no banco a partir de `data_aquisicao` pela função `notebook_lifecycle()` e exposto pela view `notebooks_with_lifecycle`, pra nunca ficar desatualizado (ver [banco-de-dados.md](#)).

Exportação de PDF do Dashboard

`lib/exportElementToPdf.ts` usa `html2canvas` pra tirar um "print" do container do Dashboard e `jspdf` pra empacotar como PDF. Antes de capturar, a classe `.is-exporting` é aplicada no container (ver `DashboardPage.tsx`) pra esconder, só na captura, o botão de exportar e eventuais mensagens de erro (via `visibility: hidden`, não `display: none`, pra não mudar o layout entre o clique e o print).

Faturas de telefonia por IA

`InvoiceUploadsPanel` faz upload do PDF direto pro Storage (bucket `phone-invoices`, com policy restrita a MASTER) e depois chama a Edge Function `process-invoice`, que:

1. confirma que quem chamou é MASTER aprovado;
2. baixa o PDF do Storage;
3. manda pro Gemini pedindo a extração estruturada (número da linha, valor, data);
4. tenta casar cada número extraído com uma linha cadastrada em `phone_lines` (`phone_line_id = null` quando não bate com nenhuma — dado real sem contrapartida no cadastro, fica registrado para revisão, não é descartado);
5. grava em `phone-invoices` com `upsert` (chave `raw_number + carrier_id + invoice_date`, pra reenviar a mesma fatura sem duplicar linhas).

Migração de dados original (Excel → Supabase)

`scripts/migration/` é o ETL que populou a base pela primeira vez a partir da planilha Excel original da BIOND. Não faz parte do runtime do app — é histórico/pontual:

```
run-etl.mjs    extrai (extract.mjs) + transforma/normaliza (transform.mjs,
               normalize.mjs, employee-registry.mjs) a planilha de data/source/,
               escreve em data/processed/ e gera um relatório em data/reports/
               (report.mjs) — não toca no Supabase.
load.mjs       lê data/processed/ e grava no Supabase (esse sim precisa da
               SERVICE_ROLE_KEY em .env.migration.local).
validate-post-load.mjs
               confere se o que foi carregado bate com o esperado.
```